DynamicServiceOrchestration

Interface Design Description

Eclipse Arrowhead 5.2.2 | core system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic. 12 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic.

DynamicServiceOrchestration - Go 5.2.2

Standalone core system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/DynamicServiceOrchestration. Entry point: ./cmd/serviceorchestration-dynamic. Module: arrowhead/ah5-core-go/serviceorchestration-dynamic. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8441 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8441". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../../LISTENING-PORTS.md) for precedence and all systems.

Detailed implementation evidence

1 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Go HTTP route inventory

This is the repository route inventory, not a new differential test. Route presence does not establish DTO, error, authorization or persistence equivalence.

- GET /serviceorchestration/general/mgmt/get-config | Go route recorded: True

- POST /serviceorchestration/general/mgmt/logs | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/history/query | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/lock/create | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/lock/query | Go route recorded: True

- DELETE /serviceorchestration/orchestration/mgmt/lock/remove/{owner} | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/push/query | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/push/subscribe | Go route recorded: True

- POST /serviceorchestration/orchestration/mgmt/push/trigger | Go route recorded: True

- DELETE /serviceorchestration/orchestration/mgmt/push/unsubscribe | Go route recorded: True

- POST /serviceorchestration/orchestration/pull | Go route recorded: True

- POST /serviceorchestration/orchestration/subscribe | Go route recorded: True

- DELETE /serviceorchestration/orchestration/unsubscribe/{id} | Go route recorded: True

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-core-rust/serviceorchestration-dynamic.

DynamicServiceOrchestration (Rust)

Arrowhead 5.2.2 core system. src/main.rs is built as binary serviceorchestration-dynamic by the arrowhead-core-rust crate in ... serviceorchestration-dynamic.json and documentation/ link to ../examples and ../documentation/DynamicServiceOrchestration.

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-generalManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-generalManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestration-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestration-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestrationLockManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestrationLockManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestrationPlantManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-sysMonitor-AMQP-IDD.tex

- Go: Eclipse-Arrowhead-go/ah5-core-go/serviceorchestration-dynamic/documentation/service-design/DynamicServiceOrchestration-PlantManagement-HTTPS-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-core-rust/documentation/DynamicServiceOrchestration/DynamicServiceOrchestration-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

generalManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `generalManagement` service interface of the DynamicServiceOrchestration system, which provides general management of the DynamicServiceOrchestration system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-log`	POST	`/logs`
`get-config`	GET	`/get-config`

2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/general/mgmt/logs

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/general/mgmt/logs HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

2.2 GET /serviceorchestration/general/mgmt/get-config?keys=*{keys}*

Operation: get-config
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /serviceorchestration/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.3 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.4 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.6 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct KeyValues . . . 6
3.5 struct LogEntry . . . 6
3.6 struct LogEntryListResponse . . . 7
3.7 struct LogRequest . . . 7
3.8 struct Page . . . 7
3.9 Primitives . . . 7

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `generalManagement` service interface of the DynamicServiceOrchestration system over MQTT, which provides general management of the DynamicServiceOrchestration system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/general/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`get-log`	`arrowhead/serviceorchestration/general/management/get-log`
`get-config`	`arrowhead/serviceorchestration/general/management/get-config`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/general/management/get-log`

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 3: Payload of an get-log request.

2.2 PUBLISH `arrowhead/serviceorchestration/general/management/get-config`

Operation: get-config
Input: Array<String>
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

```
[
  "string"
]
```

Listing 4: Payload of an get-config request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.5 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.6 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.8 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

[6] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestration HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestration"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestration` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation pull . . . 4
2.2 operation subscribe . . . 5
2.3 operation unsubscribe . . . 7

3 Data Models **8**
3.1 struct AuthorizationTokenGenerationResponse . . . 8
3.2 struct ErrorMessage . . . 8
3.3 struct MetadataRequirement . . . 8
3.4 struct OrchestrationNotifyInterface . . . 8
3.5 struct OrchestrationRequest . . . 8
3.6 struct OrchestrationResponse . . . 9
3.7 struct OrchestrationResult . . . 9
3.8 struct OrchestrationServiceRequirement . . . 9
3.9 struct OrchestrationSubscriptionRequest . . . 10
3.10 struct QoSRequirement . . . 10
3.11 struct ServiceInstanceInterfaceResponse . . . 10
3.12 Primitives . . . 11

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `serviceOrchestration` service interface of the DynamicServiceOrchestration system, which provides service orchestration: a consumer asks which provider it should use for a service, and receives the matching, authorized providers with the interfaces to call.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestration` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestration – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestration` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/orchestration`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`pull`	POST	`/pull`
`subscribe`	POST	`/subscribe`
`unsubscribe`	DELETE	`/unsubscribe/{id}`

2 Service Operations

The interfaces of the `serviceOrchestration` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestration` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/orchestration/pull

Operation: pull
Input: OrchestrationRequest
Output: OrchestrationResponse

Executes a dynamic orchestration for the requester: service discovery in the ServiceRegistry, removal of locked, blacklisted and unauthorized candidates, interface matching (optionally with translation), preferences, QoS evaluation, matchmaking, exclusive reservation and token generation. Without local results and with `ALLOW_INTERCLOUD`, or directly with `ONLY_INTERCLOUD`, the Gatekeeper is asked (local extension).

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/pull HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "serviceRequirement": {
    "serviceDefinition": "temperatureReading",
    "operations": [
      "FILTER"
    ],
    "versions": [
      "1.0.0"
    ],
    "alivesAt": "2026-10-22T12:00:00Z",
    "metadataRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "interfaceTemplateNames": [
      "string"
    ],
    "interfaceAddressTypes": [
      "string"
    ],
    "interfacePropertyRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "securityPolicies": [
      "string"
    ],
```

```
    "preferredProviders": [
      "string"
    ]
  },
  "orchestrationFlags": {
    "MATCHMAKING": true
  },
  "qualityRequirements": [
    {
      "type": "PING",
      "operation": "FILTER",
      "requirements": {
        "maxLatency": 100
      }
    }
  ],
  "exclusivityDuration": 60
}
```

Listing 1: An pull invocation.

```
{
  "results": [
    {
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "cloudIdentitifer": "LOCAL",
      "providerName": "TemperatureSensor",
      "serviceDefinitition": "temperatureReading",
      "version": "1.0.0",
      "aliveUntil": "string",
      "exclusiveUntil": "string",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ]
    }
  ],
  "warnings": [
    "string"
  ]
}
```

Listing 2: An pull response.

2.2 POST /serviceorchestration/orchestration/subscribe?trigger=*{trigger}*

Operation: subscribe
Input: OrchestrationSubscriptionRequest
Output: String

Creates (or replaces) a push subscription of the requester for itself; with `trigger=true` the orchestration is executed and pushed immediately. Returns the subscription identifier, 201 for a new subscription.

Query parameters: *trigger* (Boolean, optional, default `false`). List values are repeated or comma-separated.

A successful call returns `201 Created` for a new entry, `200 OK` if an identical entry already existed with the output in the body.

```
POST /serviceorchestration/orchestration/subscribe?trigger=<trigger> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "targetSystemName": "TemperatureDisplay",
  "orchestrationRequest": {
    "serviceRequirement": {
      "serviceDefinition": "temperatureReading",
      "operations": [
        "FILTER"
      ],
      "versions": [
        "1.0.0"
      ],
      "alivesAt": "2026-10-22T12:00:00Z",
      "metadataRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "interfaceTemplateNames": [
        "string"
      ],
      "interfaceAddressTypes": [
        "string"
      ],
      "interfacePropertyRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "securityPolicies": [
        "string"
      ],
      "preferredProviders": [
        "string"
      ]
    },
    "orchestrationFlags": {
      "MATCHMAKING": true
    },
    "qualityRequirements": [
      {
        "type": "PING",
        "operation": "FILTER",
        "requirements": {
          "maxLatency": 100
        }
      }
    ],
    "exclusivityDuration": 60
  },
  "notifyInterface": {
```

```
    "protocol": "HTTPS",
    "properties": {
      "address": "192.168.0.20",
      "port": "8080",
      "method": "POST",
      "path": "/orchestration/notify"
    }
  },
  "duration": 3600
}
```

Listing 3: An subscribe invocation.

```
"string"
```

Listing 4: An subscribe response.

2.3 DELETE /serviceorchestration/orchestration/unsubscribe/*{id}*

Operation: **unsubscribe**
Input: **String**

Removes a subscription of the requester. Returns 200 if removed, 204 if it did not exist.

Path parameter: *id*.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /serviceorchestration/orchestration/unsubscribe/<id> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An unsubscribe invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.12, which are used to represent things like hashes, identifiers and texts.

3.1 struct AuthorizationTokenGenerationResponse

A generated token.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"token"	String	The token (opaque string, Base64 or JSON Web Token).
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"expiresAt"	DateTime	UTC expiry of time-limited tokens.

3.2 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.3 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.4 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

3.5 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

3.6 struct OrchestrationResponse

The orchestration result.

Object Field	Value Type	Description
"results"	Array<OrchestrationResult>	The orchestration results, best first.
"warnings"	Array<String>	Warnings of the orchestration, e.g. `autoMatchmaking`, `notExclusive`, `interCloud`.

3.7 struct OrchestrationResult

One provider of the requested service.

Object Field	Value Type	Description
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"cloudIdentitifer"	String	Cloud identifier of the provider (`LOCAL` for the own cloud). The JSON name keeps the upstream spelling.
"providerName"	String	Name of the provider system.
"serviceDefinitition"	String	Service definition name. The JSON name keeps the upstream spelling.
"version"	Version	Version of the service instance.
"aliveUntil"	String	Time until which the service instance is registered (its expiry), if any.
"exclusiveUntil"	String	Time until which the instance is reserved exclusively for the consumer, if reserved.
"metadata"	Object	Metadata of the service instance.
"interfaces"	Array<ServiceInstanceInterface Response>	Interfaces the consumer can use (only matching ones).
"authorizationTokens"	Object<Object<AuthorizationToken GenerationResponse>>	Authorization tokens generated for the consumer, per interface template and operation.

3.8 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.9 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

3.10 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

3.11 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.12 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestration MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestration"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestration` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation pull . . . 4
2.2 operation subscribe . . . 5
2.3 operation unsubscribe . . . 6

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct AuthorizationTokenGenerationResponse . . . 7
3.5 struct MetadataRequirement . . . 8
3.6 struct OrchestrationNotifyInterface . . . 8
3.7 struct OrchestrationRequest . . . 8
3.8 struct OrchestrationResponse . . . 8
3.9 struct OrchestrationResult . . . 8
3.10 struct OrchestrationServiceRequirement . . . 9
3.11 struct OrchestrationSubscriptionRequest . . . 9
3.12 struct QoSRequirement . . . 10
3.13 struct ServiceInstanceInterfaceResponse . . . 10
3.14 Primitives . . . 10

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `serviceOrchestration` service interface of the DynamicServiceOrchestration system over MQTT, which provides service orchestration: a consumer asks which provider it should use for a service, and receives the matching, authorized providers with the interfaces to call.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestration` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestration – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/orchestration/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`pull`	`arrowhead/serviceorchestration/orchestration/pull`
`subscribe`	`arrowhead/serviceorchestration/orchestration/subscribe`
`unsubscribe`	`arrowhead/serviceorchestration/orchestration/unsubscribe`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/orchestration/pull`

Operation: pull
Input: OrchestrationRequest
Output: OrchestrationResponse

Executes a dynamic orchestration for the requester: service discovery in the ServiceRegistry, removal of locked, blacklisted and unauthorized candidates, interface matching (optionally with translation), preferences, QoS evaluation, matchmaking, exclusive reservation and token generation. Without local results and with `ALLOW_INTERCLOUD`, or directly with `ONLY_INTERCLOUD`, the Gatekeeper is asked (local extension).

```
{
  "serviceRequirement": {
    "serviceDefinition": "temperatureReading",
    "operations": [
      "FILTER"
    ],
    "versions": [
      "1.0.0"
    ],
    "alivesAt": "2026-10-22T12:00:00Z",
```

```
    "metadataRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "interfaceTemplateNames": [
      "string"
    ],
    "interfaceAddressTypes": [
      "string"
    ],
    "interfacePropertyRequirements": [
      {
        "unit": "celsius"
      }
    ],
    "securityPolicies": [
      "string"
    ],
    "preferredProviders": [
      "string"
    ]
  },
  "orchestrationFlags": {
    "MATCHMAKING": true
  },
  "qualityRequirements": [
    {
      "type": "PING",
      "operation": "FILTER",
      "requirements": {
        "maxLatency": 100
      }
    }
  ],
  "exclusivityDuration": 60
}
```

Listing 3: Payload of an pull request.

2.2 PUBLISH `arrowhead/serviceorchestration/orchestration/subscribe`

Operation: subscribe
Input: OrchestrationSubscriptionRequest
Output: String

Creates (or replaces) a push subscription of the requester for itself; with `trigger=true` the orchestration is executed and pushed immediately. Returns the subscription identifier, 201 for a new subscription.

Optional entries of `params`: `trigger`.

```
{
  "targetSystemName": "TemperatureDisplay",
  "orchestrationRequest": {
    "serviceRequirement": {
      "serviceDefinition": "temperatureReading",
      "operations": [
        "FILTER"
      ],
      "versions": [
        "1.0.0"
      ],
      "alivesAt": "2026-10-22T12:00:00Z",
      "metadataRequirements": [
        {
          "unit": "celsius"
        }
```

```
      ],
      "interfaceTemplateNames": [
        "string"
      ],
      "interfaceAddressTypes": [
        "string"
      ],
      "interfacePropertyRequirements": [
        {
          "unit": "celsius"
        }
      ],
      "securityPolicies": [
        "string"
      ],
      "preferredProviders": [
        "string"
      ]
    },
    "orchestrationFlags": {
      "MATCHMAKING": true
    },
    "qualityRequirements": [
      {
        "type": "PING",
        "operation": "FILTER",
        "requirements": {
          "maxLatency": 100
        }
      }
    ],
    "exclusivityDuration": 60
  },
  "notifyInterface": {
    "protocol": "HTTPS",
    "properties": {
      "address": "192.168.0.20",
      "port": "8080",
      "method": "POST",
      "path": "/orchestration/notify"
    }
  },
  "duration": 3600
}
```

Listing 4: Payload of an subscribe request.

2.3 PUBLISH arrowhead/serviceorchestration/orchestration/unsubscribe

Operation: unsubscribe
Input: String

Removes a subscription of the requester. Returns 200 if removed, 204 if it did not exist.

```
"3fa85f64-5717-4562-b3fc-2c963f66afa6"
```

Listing 5: Payload of an unsubscribe request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct AuthorizationTokenGenerationResponse

A generated token.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"token"	String	The token (opaque string, Base64 or JSON Web Token).
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"expiresAt"	DateTime	UTC expiry of time-limited tokens.

3.5 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.6 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

3.7 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

3.8 struct OrchestrationResponse

The orchestration result.

Object Field	Value Type	Description
"results"	Array<OrchestrationResult>	The orchestration results, best first.
"warnings"	Array<String>	Warnings of the orchestration, e.g. `autoMatchmaking`, `notExclusive`, `interCloud`.

3.9 struct OrchestrationResult

One provider of the requested service.

Object Field	Value Type	Description
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"cloudIdentitifer"	String	Cloud identifier of the provider (`LOCAL` for the own cloud). The JSON name keeps the upstream spelling.
"providerName"	String	Name of the provider system.
"serviceDefinitition"	String	Service definition name. The JSON name keeps the upstream spelling.
"version"	Version	Version of the service instance.
"aliveUntil"	String	Time until which the service instance is registered (its expiry), if any.
"exclusiveUntil"	String	Time until which the instance is reserved exclusively for the consumer, if reserved.
"metadata"	Object	Metadata of the service instance.
"interfaces"	Array<ServiceInstanceInterface Response>	Interfaces the consumer can use (only matching ones).
"authorizationTokens"	Object<Object<AuthorizationToken GenerationResponse>>	Authorization tokens generated for the consumer, per interface template and operation.

3.10 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.11 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

3.12 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

3.13 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.14 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[6] ——, “serviceOrchestration HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestration-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationHistoryManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationHistoryManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationHistoryManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct OrchestrationHistoryQueryRequest . . . 6
3.3 struct OrchestrationHistoryResponse . . . 6
3.4 struct OrchestrationJob . . . 6
3.5 struct Page . . . 7
3.6 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `serviceOrchestrationHistoryManagement` service interface of the DynamicServiceOrchestration system, which provides query of the orchestration history: the pull and push orchestration jobs, their status and outcome.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationHistoryManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationHistoryManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestrationHistoryManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/orchestration/mgmt/history`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query`	POST	`/query`

2 Service Operations

The interfaces of the `serviceOrchestrationHistoryManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestrationHistoryManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/orchestration/mgmt/history/query

Operation: query
Input: OrchestrationHistoryQueryRequest
Output: OrchestrationHistoryResponse

Returns a page of orchestration jobs matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/history/query HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ids": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ],
  "statuses": [
    "string"
  ],
  "type": "IPV4",
  "requesterSystems": [
    "TemperatureDisplay"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Listing 1: An query invocation.

```
{
```

```
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "status": "DONE",
      "type": "IPV4",
      "requesterSystem": "TemperatureDisplay",
      "targetSystem": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "subscriptionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "message": "1 local result: TemperatureSensor",
      "createdAt": "2026-10-22T12:00:00Z",
      "startedAt": "2026-10-22T12:00:00Z",
      "finishedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 2: An query response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.6, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct OrchestrationHistoryQueryRequest

Paged filter for orchestration jobs.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ids"	Array<String>	Job identifiers (UUID) to match.
"statuses"	Array<String>	Job statuses to match.
"type"	String	Orchestration type to match: `PULL` or `PUSH`.
"requesterSystems"	Array<String>	Requester system names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.
"subscriptionIds"	Array<String>	Subscription identifiers.

3.3 struct OrchestrationHistoryResponse

List of orchestration jobs.

Object Field	Value Type	Description
"entries"	Array<OrchestrationJob>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.4 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

3.5 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationHistoryManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationHistoryManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationHistoryManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationHistoryManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation query . . . 4

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct OrchestrationHistoryQueryRequest . . . 6
3.5 struct OrchestrationHistoryResponse . . . 7
3.6 struct OrchestrationJob . . . 7
3.7 struct Page . . . 7
3.8 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `serviceOrchestrationHistoryManagement` service interface of the DynamicServiceOrchestration system over MQTT, which provides query of the orchestration history: the pull and push orchestration jobs, their status and outcome.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationHistoryManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationHistoryManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/orchestration/management/history/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`query`	`arrowhead/serviceorchestration/orchestration/management/history/query`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/orchestration/management/history/query`

Operation: query
Input: OrchestrationHistoryQueryRequest
Output: OrchestrationHistoryResponse

Returns a page of orchestration jobs matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ids": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ],
  "statuses": [
    "string"
  ],
  "type": "IPV4",
  "requesterSystems": [
```

```
    "TemperatureDisplay"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Listing 3: Payload of an query request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct OrchestrationHistoryQueryRequest

Paged filter for orchestration jobs.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ids"	Array<String>	Job identifiers (UUID) to match.
"statuses"	Array<String>	Job statuses to match.
"type"	String	Orchestration type to match: `PULL` or `PUSH`.
"requesterSystems"	Array<String>	Requester system names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.
"subscriptionIds"	Array<String>	Subscription identifiers.

3.5 struct OrchestrationHistoryResponse

List of orchestration jobs.

Object Field	Value Type	Description
"entries"	Array<OrchestrationJob>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.6 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

3.7 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationHistoryManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-SD.pdf.

[6] ——, “serviceOrchestrationHistoryManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationLockManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationLockManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationLockManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query . . . 4
2.2 operation remove . . . 5
2.3 operation create . . . 5

3 Data Models **7**
3.1 struct ErrorMessage . . . 7
3.2 struct OrchestrationLockListRequest . . . 7
3.3 struct OrchestrationLockListResponse . . . 7
3.4 struct OrchestrationLockQueryRequest . . . 7
3.5 struct OrchestrationLockRequest . . . 7
3.6 struct OrchestrationLockResponse . . . 8
3.7 struct Page . . . 8
3.8 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `serviceOrchestrationLockManagement` service interface of the DynamicServiceOrchestration system, which provides operator management of orchestration locks, the exclusive reservations of service instances.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationLockManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationLockManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestrationLockManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/orchestration/mgmt/lock`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query`	POST	`/query`
`remove`	DELETE	`/remove/{owner}`
`create`	POST	`/create`

2 Service Operations

The interfaces of the `serviceOrchestrationLockManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestrationLockManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/orchestration/mgmt/lock/query

Operation: query
Input: OrchestrationLockQueryRequest
Output: OrchestrationLockListResponse

Returns a page of locks matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/lock/query HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ids": [
    1
  ],
  "orchestrationJobIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ],
  "serviceInstanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "owners": [
    "TemperatureDisplay"
  ],
  "expiresBefore": "2026-10-22T12:00:00Z",
  "expiresAfter": "2026-10-22T12:00:00Z"
}
```

Listing 1: An query invocation.

```
{
  "entries": [
    {
      "id": 1,
      "orchestrationJobId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
```

```
      "owner": "TemperatureDisplay",
      "expiresAt": "2026-10-22T12:00:00Z",
      "temporary": false
    }
  ],
  "count": 1
}
```

Listing 2: An query response.

2.2 DELETE /serviceorchestration/orchestration/mgmt/lock/remove/*{owner}*?instanceIds=*{instanceIds}*

Operation: remove
Input: String

Removes the locks of an owner on the given service instances.
Path parameter: *owner*.
Query parameters: *instanceIds* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /serviceorchestration/orchestration/mgmt/lock/remove/<owner>?instanceIds=<instanceIds> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An remove invocation.

2.3 POST /serviceorchestration/orchestration/mgmt/lock/create

Operation: create
Input: OrchestrationLockListRequest
Output: OrchestrationLockListResponse

Creates locks on service instances for owners.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/lock/create HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "locks": [
    {
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "owner": "TemperatureDisplay",
      "expiresAt": "2026-10-22T12:00:00Z"
    }
  ]
}
```

Listing 4: An create invocation.

```
{
  "entries": [
    {
      "id": 1,
      "orchestrationJobId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
```

```
      "owner": "TemperatureDisplay",
      "expiresAt": "2026-10-22T12:00:00Z",
      "temporary": false
    }
  ],
  "count": 1
}
```

Listing 5: An create response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.8, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct OrchestrationLockListRequest

Locks to create.

Object Field	Value Type	Description
"locks"	Array<OrchestrationLockRequest>	Locks to create.

3.3 struct OrchestrationLockListResponse

List of orchestration locks.

Object Field	Value Type	Description
"entries"	Array<OrchestrationLockResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.4 struct OrchestrationLockQueryRequest

Paged filter for orchestration locks.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ids"	Array<Number>	Lock identifiers to match.
"orchestrationJobIds"	Array<String>	Orchestration job identifiers to match.
"serviceInstanceIds"	Array<String>	Service instance identifiers to match.
"owners"	Array<String>	Lock owner names to match.
"expiresBefore"	DateTime	Upper bound of the expiry time.
"expiresAfter"	DateTime	Lower bound of the expiry time.

3.5 struct OrchestrationLockRequest

Exclusive reservation of a service instance.

Object Field	Value Type	Description
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"owner"	String	System for which the instance is locked.
"expiresAt"	DateTime	UTC expiry of the lock.

3.6 struct OrchestrationLockResponse

A lock on a service instance.

Object Field	Value Type	Description
"id"	Number	Identifier of the lock.
"orchestrationJobId"	String	Orchestration job that created the lock.
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"owner"	String	Name of the system owning the lock.
"expiresAt"	DateTime	UTC expiry of the lock.
"temporary"	Boolean	True for a lock held only during an orchestration.

3.7 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationLockManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationLockManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationLockManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationLockManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationLockManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation create . . . 4
2.2 operation query . . . 5
2.3 operation remove . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct OrchestrationLockListRequest . . . 6
3.5 struct OrchestrationLockListResponse . . . 6
3.6 struct OrchestrationLockQueryRequest . . . 6
3.7 struct OrchestrationLockRequest . . . 7
3.8 struct OrchestrationLockResponse . . . 7
3.9 struct Page . . . 7
3.10 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `serviceOrchestrationLockManagement` service interface of the DynamicServiceOrchestration system over MQTT, which provides operator management of orchestration locks, the exclusive reservations of service instances.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationLockManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationLockManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/orchestration/management/lock/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`create`	`arrowhead/serviceorchestration/orchestration/management/lock/create`
`query`	`arrowhead/serviceorchestration/orchestration/management/lock/query`
`remove`	`arrowhead/serviceorchestration/orchestration/management/lock/remove`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/orchestration/management/lock/create`

Operation: create
Input: OrchestrationLockListRequest
Output: OrchestrationLockListResponse

Creates locks on service instances for owners.

```
{
  "locks": [
    {
      "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "owner": "TemperatureDisplay",
      "expiresAt": "2026-10-22T12:00:00Z"
    }
  ]
}
```

Listing 3: Payload of an create request.

2.2 PUBLISH arrowhead/serviceorchestration/orchestration/management/lock/query

Operation: query
Input: OrchestrationLockQueryRequest
Output: OrchestrationLockListResponse

Returns a page of locks matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ids": [
    1
  ],
  "orchestrationJobIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ],
  "serviceInstanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "owners": [
    "TemperatureDisplay"
  ],
  "expiresBefore": "2026-10-22T12:00:00Z",
  "expiresAfter": "2026-10-22T12:00:00Z"
}
```

Listing 4: Payload of an query request.

2.3 PUBLISH arrowhead/serviceorchestration/orchestration/management/lock/remove

Operation: remove
Input: Array<String>

Removes the locks of an owner on the given service instances.

```
[
  "TemperatureDisplay"
]
```

Listing 5: Payload of an remove request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct OrchestrationLockListRequest

Locks to create.

Object Field	Value Type	Description
"locks"	Array<OrchestrationLockRequest>	Locks to create.

3.5 struct OrchestrationLockListResponse

List of orchestration locks.

Object Field	Value Type	Description
"entries"	Array<OrchestrationLockResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.6 struct OrchestrationLockQueryRequest

Paged filter for orchestration locks.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ids"	Array<Number>	Lock identifiers to match.
"orchestrationJobIds"	Array<String>	Orchestration job identifiers to match.
"serviceInstanceIds"	Array<String>	Service instance identifiers to match.
"owners"	Array<String>	Lock owner names to match.
"expiresBefore"	DateTime	Upper bound of the expiry time.
"expiresAfter"	DateTime	Lower bound of the expiry time.

3.7 struct OrchestrationLockRequest

Exclusive reservation of a service instance.

Object Field	Value Type	Description
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"owner"	String	System for which the instance is locked.
"expiresAt"	DateTime	UTC expiry of the lock.

3.8 struct OrchestrationLockResponse

A lock on a service instance.

Object Field	Value Type	Description
"id"	Number	Identifier of the lock.
"orchestrationJobId"	String	Orchestration job that created the lock.
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"owner"	String	Name of the system owning the lock.
"expiresAt"	DateTime	UTC expiry of the lock.
"temporary"	Boolean	True for a lock held only during an orchestration.

3.9 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.10 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationLockManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationLockManagement-SD.pdf.

[6] ——, “serviceOrchestrationLockManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationLockManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationPlantManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationPlantManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationPlantManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query . . . 4
2.2 operation replace . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct OrchestrationPlantProvider . . . 6
3.3 struct OrchestrationPlantRule . . . 6
3.4 struct OrchestrationPlantRules . . . 6
3.5 Primitives . . . 6

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `serviceOrchestrationPlantManagement` service interface of the Dynamic-ServiceOrchestration system, which provides management of the active engineered plant snapshot used by DynamicServiceOrchestration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationPlantManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationPlantManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestrationPlantManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/mgmt/plant-rules`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query`	POST	`/query`
`replace`	PUT	`/replace`

2 Service Operations

The interfaces of the `serviceOrchestrationPlantManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestrationPlantManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/mgmt/plant-rules/query

Operation: query
Output: OrchestrationPlantRules

Returns the current active-plant snapshot; allowed for authenticated PDE, LocalCloudManager or Sysop, also subject to the common management policy.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/mgmt/plant-rules/query HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An query invocation.

```
{
  "rules": [
    {
      "consumer": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "providers": [
        {
          "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
          "priority": 1
        }
      ]
    }
  ]
}
```

Listing 2: An query response.

2.2 PUT /serviceorchestration/mgmt/plant-rules/replace

Operation: replace
Input: OrchestrationPlantRules
Output: OrchestrationPlantRules

Atomically validates and replaces the complete snapshot. Only authenticated PDE or Sysop may replace; invalid input does not modify existing state. Empty rules clears the active plant; empty providers retains an unavailable managed scope.

A successful call returns `200 OK` with the output in the body.

```
PUT /serviceorchestration/mgmt/plant-rules/replace HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "rules": [
    {
      "consumer": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "providers": [
        {
          "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
          "priority": 1
        }
      ]
    }
  ]
}
```

Listing 3: An replace invocation.

```
{
  "rules": [
    {
      "consumer": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "providers": [
        {
          "serviceInstanceId": "TemperatureSensor|temperatureReading|1.0.0",
          "priority": 1
        }
      ]
    }
  ]
}
```

Listing 4: An replace response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.5, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct OrchestrationPlantProvider

Allowed service instance and its engineered priority.

Object Field	Value Type	Description
"serviceInstanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`.
"priority"	Number	Nonnegative rank; zero is highest and lower values win among otherwise eligible providers.

3.3 struct OrchestrationPlantRule

Managed consumer/service scope, retained even without an available provider.

Object Field	Value Type	Description
"consumer"	String	Name of the consumer system.
"serviceDefinition"	String	Service definition name.
"providers"	Array<OrchestrationPlantProvider>	Ranked allowed service instances; empty means managed but unavailable, with no unrelated or inter-cloud fallback.

3.4 struct OrchestrationPlantRules

Atomic snapshot owned by PlantDescriptionEngine at DSO.

Object Field	Value Type	Description
"rules"	Array<OrchestrationPlantRule>	Complete active-plant snapshot; an empty array clears all managed scopes.

3.5 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters,

which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationPlantManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPlantManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationPushManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationPushManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationPushManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation subscribe . . . 4
2.2 operation unsubscribe . . . 6
2.3 operation trigger . . . 6
2.4 operation query . . . 7

3 Data Models **9**
3.1 struct ErrorMessage . . . 9
3.2 struct MetadataRequirement . . . 9
3.3 struct OrchestrationJob . . . 9
3.4 struct OrchestrationNotifyInterface . . . 9
3.5 struct OrchestrationPushJobListResponse . . . 10
3.6 struct OrchestrationPushTrigger . . . 10
3.7 struct OrchestrationRequest . . . 10
3.8 struct OrchestrationServiceRequirement . . . 10
3.9 struct OrchestrationSubscriptionListRequest . . . 11
3.10 struct OrchestrationSubscriptionListResponse . . . 11
3.11 struct OrchestrationSubscriptionQueryRequest . . . 11
3.12 struct OrchestrationSubscriptionRequest . . . 11
3.13 struct OrchestrationSubscriptionResponse . . . 12
3.14 struct Page . . . 12
3.15 struct QoSRequirement . . . 12
3.16 Primitives . . . 13

4 References **14**

5 Revision History **15**
5.1 Amendments . . . 15
5.2 Quality Assurance . . . 15

1 Overview

This document describes the `serviceOrchestrationPushManagement` service interface of the Dynamic-ServiceOrchestration system, which provides operator management of push orchestration: subscriptions made on behalf of consumers, and triggering of push orchestration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationPushManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationPushManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceOrchestrationPushManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/orchestration/mgmt/push`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`subscribe`	POST	`/subscribe`
`unsubscribe`	DELETE	`/unsubscribe`
`trigger`	POST	`/trigger`
`query`	POST	`/query`

2 Service Operations

The interfaces of the `serviceOrchestrationPushManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceOrchestrationPushManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceorchestration/orchestration/mgmt/push/subscribe

Operation: subscribe
Input: OrchestrationSubscriptionListRequest
Output: OrchestrationSubscriptionListResponse

Creates subscriptions on behalf of target systems.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/subscribe HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "subscriptions": [
    {
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
```

```
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "duration": 3600
    }
  ]
}
```

Listing 1: An subscribe invocation.

```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ownerSystemName": "TemperatureDisplay",
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
```

```
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "expiredAt": "2026-10-22T12:00:00Z",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An subscribe response.

2.2 DELETE /serviceorchestration/orchestration/mgmt/push/unsubscribe?ids={ids}

Operation: unsubscribe
Input: Array<String>

Removes subscriptions by identifier.
Query parameters: *ids* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /serviceorchestration/orchestration/mgmt/push/unsubscribe?ids=<ids> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An unsubscribe invocation.

2.3 POST /serviceorchestration/orchestration/mgmt/push/trigger

Operation: trigger
Input: OrchestrationPushTrigger
Output: OrchestrationPushJobListResponse

Starts push orchestration for the selected subscriptions (by target system or identifier); returns the started jobs.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/trigger HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Listing 4: An trigger invocation.

```
{
  "jobs": [
    {
```

```
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "status": "DONE",
      "type": "IPV4",
      "requesterSystem": "TemperatureDisplay",
      "targetSystem": "TemperatureDisplay",
      "serviceDefinition": "temperatureReading",
      "subscriptionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "message": "1 local result: TemperatureSensor",
      "createdAt": "2026-10-22T12:00:00Z",
      "startedAt": "2026-10-22T12:00:00Z",
      "finishedAt": "2026-10-22T12:00:00Z"
    }
  ]
}
```


Listing 5: An trigger response.

2.4 POST /serviceorchestration/orchestration/mgmt/push/query

Operation: query
Input: OrchestrationSubscriptionQueryRequest
Output: OrchestrationSubscriptionListResponse

Returns a page of subscriptions matching the filter.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/orchestration/mgmt/push/query HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ownerSystems": [
    "string"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ]
}
```

Listing 6: An query invocation.


```
{
  "entries": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "ownerSystemName": "TemperatureDisplay",
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
          "operations": [
            "FILTER"
          ],
          "versions": [
```

```
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "expiredAt": "2026-10-22T12:00:00Z",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 7: An query response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.16, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.3 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

3.4 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

3.5 struct OrchestrationPushJobListResponse

Push jobs started by a trigger.

Object Field	Value Type	Description
"jobs"	Array<OrchestrationJob>	Orchestration jobs started by the trigger.

3.6 struct OrchestrationPushTrigger

Selects the subscriptions to execute now.

Object Field	Value Type	Description
"targetSystems"	Array<String>	Consumers whose subscriptions are executed.
"subscriptionIds"	Array<String>	Subscriptions to execute.

3.7 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

3.8 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.9 struct OrchestrationSubscriptionListRequest

Subscriptions to create on behalf of consumers.

Object Field	Value Type	Description
"subscriptions"	Array<OrchestrationSubscription Request>	Subscriptions to create.

3.10 struct OrchestrationSubscriptionListResponse

List of subscriptions.

Object Field	Value Type	Description
"entries"	Array<OrchestrationSubscription Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.11 struct OrchestrationSubscriptionQueryRequest

Paged filter for subscriptions.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ownerSystems"	Array<String>	Subscription owner names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.

3.12 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

3.13 struct OrchestrationSubscriptionResponse

A stored subscription.

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the subscription.
"ownerSystemName"	String	Name of the system that created the subscription.
"targetSystemName"	String	Name of the consumer system the push orchestration is made for.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"expiredAt"	DateTime	UTC time when the subscription expires.
"createdAt"	DateTime	UTC time of creation.

3.14 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.15 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

3.16 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationPushManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceOrchestrationPushManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceOrchestrationPushManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceOrchestrationPushManagement` service of the DynamicServiceOrchestration system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation subscribe . . . 4
2.2 operation trigger . . . 5
2.3 operation unsubscribe . . . 6
2.4 operation query . . . 6

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct MetadataRequirement . . . 7
3.5 struct OrchestrationJob . . . 7
3.6 struct OrchestrationNotifyInterface . . . 8
3.7 struct OrchestrationPushJobListResponse . . . 8
3.8 struct OrchestrationPushTrigger . . . 8
3.9 struct OrchestrationRequest . . . 8
3.10 struct OrchestrationServiceRequirement . . . 9
3.11 struct OrchestrationSubscriptionListRequest . . . 9
3.12 struct OrchestrationSubscriptionListResponse . . . 9
3.13 struct OrchestrationSubscriptionQueryRequest . . . 9
3.14 struct OrchestrationSubscriptionRequest . . . 10
3.15 struct OrchestrationSubscriptionResponse . . . 10
3.16 struct Page . . . 10
3.17 struct QoSRequirement . . . 10
3.18 Primitives . . . 11

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `serviceOrchestrationPushManagement` service interface of the Dynamic-ServiceOrchestration system over MQTT, which provides operator management of push orchestration: subscriptions made on behalf of consumers, and triggering of push orchestration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceOrchestrationPushManagement` service interface

This document provides the Interface Design Description IDD to the *serviceOrchestrationPushManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceorchestration/orchestration/management/push/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`subscribe`	`arrowhead/serviceorchestration/orchestration/management/push/subscribe`
`trigger`	`arrowhead/serviceorchestration/orchestration/management/push/trigger`
`unsubscribe`	`arrowhead/serviceorchestration/orchestration/management/push/unsubscribe`
`query`	`arrowhead/serviceorchestration/orchestration/management/push/query`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceorchestration/orchestration/management/push/subscribe`

Operation: subscribe
Input: OrchestrationSubscriptionListRequest
Output: OrchestrationSubscriptionListResponse

Creates subscriptions on behalf of target systems.

```
{
  "subscriptions": [
    {
      "targetSystemName": "TemperatureDisplay",
      "orchestrationRequest": {
        "serviceRequirement": {
          "serviceDefinition": "temperatureReading",
```

```
          "operations": [
            "FILTER"
          ],
          "versions": [
            "1.0.0"
          ],
          "alivesAt": "2026-10-22T12:00:00Z",
          "metadataRequirements": [
            {}
          ],
          "interfaceTemplateNames": [
            "string"
          ],
          "interfaceAddressTypes": [
            "string"
          ],
          "interfacePropertyRequirements": [
            {}
          ],
          "securityPolicies": [
            "string"
          ],
          "preferredProviders": [
            "string"
          ]
        },
        "orchestrationFlags": {
          "MATCHMAKING": true
        },
        "qualityRequirements": [
          {}
        ],
        "exclusivityDuration": 60
      },
      "notifyInterface": {
        "protocol": "HTTPS",
        "properties": {
          "address": "192.168.0.20",
          "port": "8080",
          "method": "POST",
          "path": "/orchestration/notify"
        }
      },
      "duration": 3600
    }
  ]
}
```

Listing 3: Payload of an subscribe request.

2.2 PUBLISH arrowhead/serviceorchestration/orchestration/management/push/trigger

Operation: trigger
Input: OrchestrationPushTrigger
Output: OrchestrationPushJobListResponse

Starts push orchestration for the selected subscriptions (by target system or identifier); returns the started jobs.

```
{
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "subscriptionIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
```

```
}
```

Listing 4: Payload of an trigger request.

2.3 PUBLISH arrowhead/serviceorchestration/orchestration/management/push/unsubscribe

Operation: unsubscribe
Input: Array<String>

Removes subscriptions by identifier.

```
[
  "3fa85f64-5717-4562-b3fc-2c963f66afa6"
]
```

Listing 5: Payload of an unsubscribe request.

2.4 PUBLISH arrowhead/serviceorchestration/orchestration/management/push/query

Operation: query
Input: OrchestrationSubscriptionQueryRequest
Output: OrchestrationSubscriptionListResponse

Returns a page of subscriptions matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "ownerSystems": [
    "string"
  ],
  "targetSystems": [
    "TemperatureDisplay"
  ],
  "serviceDefinitions": [
    "temperatureReading"
  ]
}
```

Listing 6: Payload of an query request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.5 struct OrchestrationJob

One orchestration job (pull or push).

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the orchestration job.
"status"	String	Status of the job: `PENDING`, `IN_PROGRESS`, `DONE` or `ERROR`.
"type"	String	Orchestration type: `PULL` or `PUSH`.
"requesterSystem"	String	Name of the system that requested the orchestration.
"targetSystem"	String	Name of the consumer system the orchestration was made for.
"serviceDefinition"	String	Service definition name.
"subscriptionId"	String	Subscription that started the push job.
"message"	String	Status message, e.g. `1 local result: DataManager`.
"createdAt"	DateTime	UTC time of creation.
"startedAt"	DateTime	UTC time when the job started.
"finishedAt"	DateTime	UTC time when the job finished.

3.6 struct OrchestrationNotifyInterface

Notification endpoint of push orchestration.

Object Field	Value Type	Description
"protocol"	String	Protocol of the notification: `HTTP`, `HTTPS`, `MQTT` or `MQTTS`.
"properties"	Object<String>	For HTTP(S) `address`, `port`, `method`, `path`; for MQTT(S) `topic` (and broker `address`, `port`).

3.7 struct OrchestrationPushJobListResponse

Push jobs started by a trigger.

Object Field	Value Type	Description
"jobs"	Array<OrchestrationJob>	Orchestration jobs started by the trigger.

3.8 struct OrchestrationPushTrigger

Selects the subscriptions to execute now.

Object Field	Value Type	Description
"targetSystems"	Array<String>	Consumers whose subscriptions are executed.
"subscriptionIds"	Array<String>	Subscriptions to execute.

3.9 struct OrchestrationRequest

An orchestration request.

Object Field	Value Type	Description
"serviceRequirement"	OrchestrationServiceRequirement	Requirements on the service the consumer wants to use.
"orchestrationFlags"	Object<Boolean>	Orchestration flags, e.g. `MATCHMAKING`, `ONLY_PREFERRED`, `ALLOW_INTERCLOUD`.
"qualityRequirements"	Array<QoSRequirement>	Quality-of-service requirements (evaluated by DeviceQoSEvaluator).
"exclusivityDuration"	Number	Requested exclusive reservation of the provider in seconds.

3.10 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.11 struct OrchestrationSubscriptionListRequest

Subscriptions to create on behalf of consumers.

Object Field	Value Type	Description
"subscriptions"	Array<OrchestrationSubscription Request>	Subscriptions to create.

3.12 struct OrchestrationSubscriptionListResponse

List of subscriptions.

Object Field	Value Type	Description
"entries"	Array<OrchestrationSubscription Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.13 struct OrchestrationSubscriptionQueryRequest

Paged filter for subscriptions.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"ownerSystems"	Array<String>	Subscription owner names to match.
"targetSystems"	Array<String>	Target system names.
"serviceDefinitions"	Array<String>	Service definition names to match.

3.14 struct OrchestrationSubscriptionRequest

A push orchestration subscription.

Object Field	Value Type	Description
"targetSystemName"	String	Consumer system; default the requester.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"duration"	Number	Requested lifetime of the subscription in seconds; no value means unlimited.

3.15 struct OrchestrationSubscriptionResponse

A stored subscription.

Object Field	Value Type	Description
"id"	String	Identifier (UUID) of the subscription.
"ownerSystemName"	String	Name of the system that created the subscription.
"targetSystemName"	String	Name of the consumer system the push orchestration is made for.
"orchestrationRequest"	OrchestrationRequest	The orchestration request executed for the subscription.
"notifyInterface"	OrchestrationNotifyInterface	Where and how the orchestration result is pushed to the target system.
"expiredAt"	DateTime	UTC time when the subscription expires.
"createdAt"	DateTime	UTC time of creation.

3.16 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.17 struct QoSRequirement

One quality-of-service requirement.

Object Field	Value Type	Description
"type"	String	Type of the QoS evaluation, as known by DeviceQoSEvaluator.
"operation"	String	QoS operation: `FILTER` or `SORT`.
"requirements"	Object	Parameters of the QoS requirement.

3.18 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceOrchestrationPushManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-SD.pdf.

[6] ——, “serviceOrchestrationPushManagement HTTP/TLS/JSON – Interface Design Description (DynamicServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the DynamicServiceOrchestration system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration 3

2 Service Operations **4**
2.1 operation echo 4
2.2 operation power 4
2.3 operation status 4

3 Data Models **6**
3.1 struct Observation 6
3.2 Primitives 6

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of DynamicServiceOrchestration.
It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, DynamicServiceOrchestration registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.DynamicServiceOrchestration.
    DynamicServiceOrchestration",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher connects to the broker (`sysmonitor.broker.address`, default `amqp-broker`; port `sysmonitor.broker.port`, default 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. The local cloud name is taken from `sysmonitor.local.cloud` or the system certificate. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.DynamicServiceOrchestration.DynamicServiceOrchestration.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.DynamicServiceOrchestration.DynamicServiceOrchestration.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.DynamicServiceOrchestration.DynamicServiceOrchestration.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "DynamicServiceOrchestration-echo-1792670400000-42",
  "providerSystem": "DynamicServiceOrchestration",
  "serviceInstance": "DynamicServiceOrchestration",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "DynamicServiceOrchestration is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
```

```
18 }
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power` or `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 System context

DynamicServiceOrchestration discovers providers through ServiceRegistry, verifies consumers through ConsumerAuthorization and returns eligible interfaces. PDE supplies engineering constraints through a separate snapshot API. The implementation shares the native Go core runtime; no Java or Rust process implements it. Existing supported dynamic orchestration and optional SimpleStore services retain their separate identities and state.

2 PDE and DSO snapshot extension (23 October 2026)

The local Java, Go and native Rust implementations provide the same HTTPS snapshot contract. The registered service is `serviceOrchestrationPlantManagement`, version 1.0.0. The API is a local extension to the upstream framework.

Method	Resource and operation
POST	`/serviceorchestration/mgmt/plant-rules/query` (query)
PUT	`/serviceorchestration/mgmt/plant-rules/replace` (replace)

Both operations return the complete snapshot. The request for replacement has `rules`, an array of scopes. Each scope contains `consumer`, `serviceDefinition`, and `providers`; each provider contains `serviceInstanceId` and integer `priority`. Priority zero is valid; lower values rank first. Duplicate scopes and provider IDs, null arrays, and invalid priorities are rejected before commit. Empty rules clear management; empty providers retain a managed but unavailable scope.

PDE composes includes and resolves names/metadata against registered systems and instances. Named consumers remain managed when unregistered; missing or ambiguous producers retain empty scopes. Repeated provider IDs use the minimum priority. PDE queries before replacing and requires an equal acknowledgment. Failed reads do not clear rules. Registry query pages are bounded at 1000 entries. An incomplete response fails synchronization and preserves the last DSO snapshot. Changes reconcile normally within two seconds of a plant change and within thirty seconds of registry changes. `/plantdescriptionengine/mgmt/sync-status` exposes the loop state under management access.

DSO restricts candidates to the scope, retains existing authorization, operation/interface and exclusivity checks, then ranks eligible providers. Matchmaking selects the lowest eligible priority. A denied or missing primary can use an engineered fallback. Unrelated providers and both inter-cloud flags cannot escape a managed scope. Unsupported existing translation/QoS features remain unsupported.

Mutual TLS and certificate-bound identity tokens remain mandatory. Only PDE or Sysop can replace; PDE, LocalCloudManager or Sysop can query. PDE gains no general DSO administration right. ServiceRegistry reserves the new service definition for DSO. Native Rust permits PDE only the two required registry query operations. DSO's `plantRules.active` record is persisted by an atomic private JSON commit and survives restart. Existing native state files need no SQL migration. Java uses its separate table migration.

The Go deployment script regenerates pinned runtime origins and installs scoped policies; the Rust script targets the native candidate. Both preserve state. Legacy SimpleStore rows remain independent and are not imported or automatically deleted. Coordinated rollback must restore PDE and its backend together. Current live evidence covers priority zero, fallback, unavailable producers, both inter-cloud guards, restart persistence with PDE stopped, and deactivation. This does not claim complete Java equivalence or production load qualification.

1 Interface overview

Mandatory mutual TLS, Arrowhead token/certificate binding, JSON request and response documents. Echo responses, orchestration subscription IDs, authorization public keys and encryption IV responses are text. Certificate-only operations and management are distinguished in the table.

1.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/serviceorchestration/general/mgmt/get-config	management
POST	/serviceorchestration/general/mgmt/logs	management
POST	/serviceorchestration/orchestration/pull	identity
POST	/serviceorchestration/orchestration/subscribe	identity
DELETE	/serviceorchestration/orchestration/unsubscribe/{id}	identity
POST	/serviceorchestration/orchestration/mgmt/push/subscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/trigger	management
DELETE	/serviceorchestration/orchestration/mgmt/push/unsubscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/query	management
POST	/serviceorchestration/orchestration/mgmt/history/query	management
POST	/serviceorchestration/mgmt/plant-rules/query	management
PUT	/serviceorchestration/mgmt/plant-rules/replace	management
POST	/serviceorchestration/orchestration/mgmt/lock/create	management
POST	/serviceorchestration/orchestration/mgmt/lock/query	management
DELETE	/serviceorchestration/orchestration/mgmt/lock/remove/{owner}	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

2 Limitations

Local/inter-cloud orchestration, authorization, operation filtering, tokens, preferences, matchmaking, locks, history and HTTPS push are implemented. The local PDE extension adds atomic persistent plant scopes and ranked providers; managed scopes cannot escape through inter-cloud flags. Translation, QoS, complete blacklist pipelines, registry-event-driven reevaluation and full Java parity remain unqualified.

3 PDE and DSO snapshot extension (23 October 2026)

The local Java, Go and native Rust implementations provide the same HTTPS snapshot contract. The registered service is `serviceOrchestrationPlantManagement`, version 1.0.0. The API is a local extension to the upstream framework.

Method	Resource and operation
POST	`/serviceorchestration/mgmt/plant-rules/query` (query)
PUT	`/serviceorchestration/mgmt/plant-rules/replace` (replace)

Both operations return the complete snapshot. The request for replacement has `rules`, an array of scopes. Each scope contains `consumer`, `serviceDefinition`, and `providers`; each provider contains `serviceInstanceId` and integer `priority`. Priority zero is valid; lower values rank first. Duplicate scopes and provider IDs, null arrays, and invalid priorities are rejected before commit. Empty rules clear management; empty providers retain a managed but unavailable scope.

PDE composes includes and resolves names/metadata against registered systems and instances. Named consumers remain managed when unregistered; missing or ambiguous producers retain empty scopes. Repeated provider IDs use the minimum priority. PDE queries before replacing and requires an equal acknowledgment. Failed reads do not clear rules. Registry query pages are bounded at 1000 entries. An incomplete response fails synchronization and preserves the last DSO snapshot. Changes reconcile normally within two seconds of a plant change and within thirty seconds of registry changes. `/plantdescriptionengine/mgmt/sync-status` exposes the loop state under management access.

DSO restricts candidates to the scope, retains existing authorization, operation/interface and exclusivity checks, then ranks eligible providers. Matchmaking selects the lowest eligible priority. A denied or missing primary can use an engineered fallback. Unrelated providers and both inter-cloud flags cannot escape a managed scope. Unsupported existing translation/QoS features remain unsupported.

Mutual TLS and certificate-bound identity tokens remain mandatory. Only PDE or Sysop can replace; PDE, LocalCloudManager or Sysop can query. PDE gains no general DSO administration right. ServiceRegistry reserves the new service definition for DSO. Native Rust permits PDE only the two required registry query operations. DSO's `plantRules.active` record is persisted by an atomic private JSON commit and survives restart. Existing native state files need no SQL migration. Java uses its separate table migration.

The Go deployment script regenerates pinned runtime origins and installs scoped policies; the Rust script targets the native candidate. Both preserve state. Legacy SimpleStore rows remain independent and are not imported or automatically deleted. Coordinated rollback must restore PDE and its backend together. Current live evidence covers priority zero, fallback, unavailable producers, both inter-cloud guards, restart persistence with PDE stopped, and deactivation. This does not claim complete Java equivalence or production load qualification.